HouseCallPro Integration

1. Overview

1.1 Description

HouseCallPro Integration connects **HouseCallPro** (field service management platform) with **Newo Platform**.

It enables:

- Checking available appointment slots with intelligent employee selection (by role, job type, or zip code)
- Booking appointments with automatic customer search, creation, address comparison, and job management
- Cancelling existing appointments
- Recognizing returning customers on inbound calls with upcoming job context
- Caching company data (employees, job types, locations) with configurable refresh intervals
- Automatic line item matching using AI for job cost estimation

This integration uses **custom tooling** — tools are registered via `project_attributes_settings_newo_tools` and triggered by agent code-phrases, receiving conversation memory for context extraction.

1.2 Use Cases

- When a customer asks about available slots -> AI selects the right employee and returns open time slots
- When a customer wants to book a service -> Search or create customer record, create job with line items, and schedule appointment
- When a customer wants to cancel -> Cancel the appointment via HouseCallPro API
- When an inbound call arrives -> Automatically look up customer by phone and display upcoming jobs to the agent
- When a customer describes a service need -> AI matches the description to job types and line items for accurate quoting
- When a session starts -> Static data (employees, job types, company info) is refreshed if cache is stale

1.3 Supported Features

Feature	Supported	Notes
Check Availability	Yes	Single API call with employee_ids for server-side filtering
Book Appointment	Yes	Creates job + appointment with customer management
Cancel Appointment	Yes	Uses booking_id/job_id from persona attributes
Employee Selection (POC)	Yes	Assigns Point of Contact employee (default method)
Employee Selection (Job Type)	Yes	AI matches job description to employee skills/tags

Employee Selection (Zip Code)	Yes	Matches employee by service area zip code tags
Customer Search	Yes	By phone number via HouseCallPro API
Customer Creation	Yes	Auto-creates during booking flow with address
Address Comparison	Yes	Compares provided address with HCP profile, adds new address if different
Existing Client Recognition	Yes	Looks up customer + upcoming jobs on inbound call
Auto Line Item Matching	Yes	AI matches service description to catalog items
Static Data Caching	Yes	Employees, job types, company info with 24h TTL
Multi-Location Support	Yes	Location-specific employee filtering
Token Auto-Refresh	No	Uses static API key (Bearer token)
Reschedule Appointment	No	Cancel + rebook as workaround
Historical Data Import	No	Only new events after activation

2. Requirements

Before installation:

- Active **HouseCallPro** account (Pro plan or higher for API access)
- **HouseCallPro API key** for authentication
- At least one employee configured with appropriate permissions
- For POC method: at least one employee marked as Point of Contact in HouseCallPro

3. How to Setup

3.1 Step 1 — Get API Credentials

1. Log in to your **HouseCallPro** account
2. Navigate to account settings or developer/API section
3. Generate or locate your **API key**
4. Note your company timezone setting

3.2 Step 2 — Connect in Platform

1. Open **Newo -> Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Description
`hcp_api_key`	Yes	HouseCallPro API key for authentication
`hcp_base_url`	No	API base URL (default: `https://api.housecallpro.com/`)
`hcp_enable_slot_check`	No	Enable availability checks (default: `True`)

`hcp_enable_booking`	No	Enable appointment creation (default: `True`)
`hcp_enable_cancellation`	No	Enable cancellations (default: `True`)
`hcp_auto_employee_type`	No	Employee assignment method: `poc`, `job_type`, or `zip_code` (default: `poc`)
`hcp_booking_duration`	No	Appointment duration in minutes (default: `120`; legacy values `1..12` are interpreted as hours)
`hcp_show_for_days`	No	Days to search for availability (default: `5`)
`hcp_availability_offset`	No	Days before first available slot (default: `2`)
`hcp_check_existing_client`	No	Look up customer on inbound call (default: `True`)
`hcp_auto_add_line_items`	No	Auto-match line items to jobs via AI (default: `False`)
`hcp_line_items`	No	Service catalog as JSON array of items
`hcp_job_type_employee_selection_prompt`	No	Custom LLM prompt for job-type employee matching
`hcp_static_data_update_interval`	No	Cache refresh interval in seconds (default: `86400`)

3. Click **Save + Publish All**
4. On publish, the InitFlow automatically:
    - Creates an HTTP connector for all API operations
    - Registers 3 custom tools (`hcp_check_availability` , `hcp_book_appointment` , `hcp_cancel_appointment`)
    - Fetches and caches company info, employees, and job types
    - Sets up booking and cancellation scenarios on the canvas

4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

4.1 How the Integration Works

The integration uses **custom tooling** instead of standard NAF booking tools. Three custom tools are registered in `project_attributes_settings_newo_tools` and triggered by agent code-phrases:

Custom Tool	Code-phrase trigger	Purpose
`hcp_check_availability`	"Let me check available time slots for you..."	Check available appointment slots
`hcp_book_appointment`	"I'm booking your appointment now..."	Book an appointment
`hcp_cancel_appointment`	"I'm cancelling your appointment now..."	Cancel an appointment

Custom tools receive `arguments.memory` (conversation history) instead of structured payload. NSLG extraction skills use LLM to extract structured parameters from the conversation context.

Results are returned via `urgent_message` events (not `result_success` / `result_error`).

Available Triggers

Trigger	Event	Description
Check Availability	`hcp_check_availability`	Custom tool: agent checks available slots
Book Appointment	`hcp_book_appointment`	Custom tool: agent books an appointment
Cancel Appointment	`hcp_cancel_appointment`	Custom tool: agent cancels an appointment
Session Start	`session_started`	Refreshes static data cache if stale
Inbound Call	`define_user_phone_number`	Looks up existing customer and upcoming jobs
Setup	`project_publish_finish`	Initializes integration on deploy
Session End	`end_session`	Clears temporary customer data

Available Actions

- **Get Company** — Fetch company info and locations (GET `/company`)
- **Get Job Types** — Fetch available service types (extracted from company response)
- **Get Employees** — Fetch employees, optionally by location (GET `/employees`)
- **Get Booking Windows** — Fetch available time slots with employee filtering (GET `/company/schedule_availability/booking_windows?employee_ids[]=...`)
- **Search Customer** — Find existing customer by phone (GET `/customers`)
- **Create Customer** — Create a new customer record (POST `/customers`)
- **Add Customer Address** — Add a new address to an existing customer (POST `/customers/{id}/addresses`)
- **Create Job** — Create a service job (POST `/jobs`)
- **Create Appointment** — Schedule a time slot on a job (POST `/jobs/{id}/appointments`)
- **Cancel Appointment** — Delete a scheduled appointment (DELETE `/jobs/{id}/appointments/{apt_id}`)
- **Get Job Appointments** — Fetch appointments for a job (GET `/jobs/{id}/appointments`)

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as HCP Integration
    participant API as HouseCallPro API

    Note over I,API: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: GET /company
    I->>API: GET /employees
```

```
I->>A: Custom tools registered, data cached

Note over I,API: Inbound Call
A->>I: define_user_phone_number
I->>API: GET /customers?q={phone}
API-->>I: Customer data
I->>API: GET /jobs?customer_id={id}
API-->>I: Upcoming jobs
I->>A: ExistingCustomerInfo injected

Note over U,API: Availability Check (custom tool, parallel per-employee)
U->>A: "Any slots this week?"
A->>A: Code-phrase: "Let me check available time slots..."
A->>I: hcp_check_availability (memory)
I->>I: Extract params from memory (NSLG)
I->>I: Select employees (POC/job_type/zip), max 5
par Parallel requests per employee
    I->>API: GET /booking_windows?employee_ids[]=emp1
    I->>API: GET /booking_windows?employee_ids[]=emp2
end
API-->>I: Per-employee windows
I->>I: Merge slots, save slot→employee map
I->>A: urgent_message (all available slots)
A->>U: "Here are the available times..."

Note over U,API: Booking (custom tool, with double-booking protection)
U->>A: "Book April 3 at 9 AM"
A->>A: Code-phrase: "I'm booking your appointment now..."
A->>I: hcp_book_appointment (memory)
I->>I: Check hcp_booking_in_progress lock
I->>I: Extract params from memory (NSLG)
I->>I: Resolve employee from slot→employee map
I->>API: GET /customers?q={phone}
alt Customer not found
    I->>API: POST /customers
end
I->>API: POST /jobs
I->>API: POST /jobs/{id}/appointments (with mapped employee)
API-->>I: appointment_id
I->>I: Save booking_id/job_id to persona, release lock
I->>A: urgent_message (booking confirmed)
A->>U: "Your appointment is confirmed!"

Note over U,API: Cancellation (custom tool)
U->>A: "Cancel my appointment"
A->>A: Code-phrase: "I'm cancelling your appointment now..."
A->>I: hcp_cancel_appointment (memory)
I->>I: Read booking_id/job_id from persona
I->>API: DELETE /jobs/{job_id}/appointments/{booking_id}
API-->>I: Confirmation
```

```
    I->>A: urgent_message (cancelled)
    A->>U: "Your appointment has been cancelled"
```

StaticDataFlow (Caching)

```
flowchart TD
    E["session_started /<br>hcp_update_static_data"] --> CHK["Check cache age:
<br>updated_at vs now"]
    CHK --> STALE{"Cache older than<br>update_interval?"}
    STALE -->|"No"| SKIP["Use cached data"]
    STALE -->|"Yes"| COMP["GET /company<br>Extract locations, timezone"]
    COMP --> TYPES["Extract job_types<br>from company response"]
    TYPES --> LOC{"Multiple<br>locations?"}
    LOC -->|"No"| EMP["GET /employees?page_size=100"]
    LOC -->|"Yes"| EMP_LOC["GET /employees<br>per location_id"]
    EMP --> SAVE["Save to hcp_company_employees<br>Update timestamp"]
    EMP_LOC --> SAVE_LOC["Save to hcp_location_employees<br>Update timestamp"]
```

AvailabilityFlow (Parallel Per-Employee)

```
flowchart TD
    E["hcp_check_availability<br>(custom tool)"] --> CHK{"Slot check<br>enabled?"}
    CHK -->|"No"| SKIP["Return (disabled)"]
    CHK -->|"Yes"| EXTRACT["Extract from memory (NSLG):<br>job_description,
zip_code, date_time"]
    EXTRACT --> METHOD{"Employee<br>selection method?"}

    METHOD -->|"poc"| POC["Filter employees:<br>is_point_of_contact = true"]
    METHOD -->|"job_type"| JT["LLM matches job_description<br>to employee
tags/skills<br>(Gemini 2.5 Flash)"]
    METHOD -->|"zip_code"| ZIP["Filter employees:<br>tags contain customer zip"]

    JT --> FALLBACK{"Employees<br>found?"}
    ZIP --> FALLBACK
    FALLBACK -->|"No"| POC_FB["Fallback: POC"]
    POC_FB --> POC_FB2{"POC found?"}
    POC_FB2 -->|"No"| ALL["Fallback: all<br>can_be_booked_online"]
    POC_FB2 -->|"Yes"| LIMIT
    ALL --> LIMIT
    FALLBACK -->|"Yes"| LIMIT
    POC --> LIMIT

    LIMIT["Take top 5 employees"]
    LIMIT --> PARALLEL["Send N parallel requests<br>(one per employee)"]
    PARALLEL --> REQ1["GET /booking_windows<br>?employee_ids[]=emp1"]
    PARALLEL --> REQ2["GET /booking_windows<br>?employee_ids[]=emp2"]
    PARALLEL --> REQN["..."]

    REQ1 --> COLLECT["_collectEmployeeSlotsSkill<br>Store in hcp_slots_{emp_id}<br>
(separate attribute per employee)"]
```

```
    REQ2 --> COLLECT
    REQN --> COLLECT

    COLLECT --> COUNTER{"All employees<br>collected?"}
    COUNTER -->|"No"| WAIT["Wait for remaining"]
    COUNTER -->|"Yes"| MERGE["_getEmployeeSlotsSkill:<br>Merge all slots,<br>sort by
time,<br>save slot→employee map"]
    MERGE --> OUT["urgent_message<br>{all availability slots}"]
```

BookingFlow (Custom Tool)

```
flowchart TD
    E["hcp_book_appointment<br>(custom tool)"] --> CHK{"Booking<br>enabled?"}
    CHK -->|"No"| SKIP["Return (disabled)"]
    CHK -->|"Yes"| LOCK{"hcp_booking_in_progress<br>= true?"}
    LOCK -->|"Yes"| DUP["Skip (duplicate)"]
    LOCK -->|"No"| SETLOCK["Set lock = true"]
    SETLOCK --> EXTRACT["Extract from memory (NSLG):<br>customerInfo, address,
<br>bookingDateTime, notes"]
    EXTRACT --> PHONE["Format phone:<br>strip +, remove leading 1"]
    PHONE --> SEARCH["GET /customers?q={phone}"]
    SEARCH --> FOUND{"Customer<br>found?"}
    FOUND -->|"Yes"| ADDR{"Address matches?"}
    FOUND -->|"No"| CREATE["POST /customers"]
    CREATE --> NEW["Extract customer_id<br>+ address_id"]
    ADDR -->|"Yes"| USE["Use existing<br>address_id"]
    ADDR -->|"No"| ADD["POST /customers/{id}/addresses"]
    ADD --> NOTE["Append 'Address updated'<br>to job notes"]
    NOTE --> JOB
    USE --> JOB
    NEW --> JOB
    JOB["POST /jobs<br>{customer_id, address_id, notes}"]
    JOB --> LINE{"Auto line items?"}
    LINE -->|"Yes"| LLM["LLM matches description<br>to catalog + job_types"]
    LINE -->|"No"| EMP
    LLM --> EMP
    EMP["Resolve employee from<br>hcp_slot_employee_map<br>(fallback:
selected_employee_id)"]
    EMP --> APPT["POST /jobs/{id}/appointments<br>(with mapped employee)"]
    APPT --> SAVE["Save booking_id + job_id<br>to persona, release lock"]
    SAVE --> OK{"Success?"}
    OK -->|"No"| ERR["urgent_message (error),<br>release lock"]
    OK -->|"Yes"| OUT["urgent_message<br>(booking confirmed)"]
```

CancellationFlow (Custom Tool)

```
flowchart TD
    E["hcp_cancel_appointment<br>(custom tool)"] --> CHK{"Cancellation<br>enabled?"}
    CHK -->|"No"| SKIP["Return (disabled)"]
    CHK -->|"Yes"| IDS["Read from persona:
```

```
<br>hcp_last_booking_id<br>hcp_last_job_id"]
    IDS --> VAL{"IDs present?"}
    VAL -->|"No"| ERR["urgent_message:<br>no appointment found"]
    VAL -->|"Yes"| API["DELETE /jobs/{job_id}<br>/appointments/{booking_id}"]
    API --> OK{"Success?"}
    OK -->|"No"| ERR2["urgent_message (error)"]
    OK -->|"Yes"| OUT["urgent_message<br>(cancelled)"]
```

ExistingClientFlow (Caller Recognition)

```
flowchart TD
    E["define_user_phone_number"] --> CHK{"Check existing<br>client enabled?"}
    CHK -->|"No"| SKIP["Skip lookup"]
    CHK -->|"Yes"| SEARCH["GET /customers?q={phone}"]
    SEARCH --> FOUND{"Customer<br>found?"}
    FOUND -->|"No"| DONE["No context injected"]
    FOUND -->|"Yes"| INFO["Extract: name, email,<br>phone, address"]
    INFO --> JOBS["GET /jobs?customer_id={id}<br>&scheduled_start_min={today}"]
    JOBS --> APPTS["For each job:<br>GET /jobs/{id}/appointments"]
    APPTS --> INJECT["Inject ExistingCustomerInfo<br>into agent context"]
    INJECT --> DONE2["Agent sees customer<br>history on greeting"]
```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (voice or chat).

4.4 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify static data is cached (check `hcp_company_employees` is populated)
2. **Custom Tools**: Verify `project_attributes_settings_newo_tools` contains 3 tools after publish
3. **Static Data**: Verify `hcp_company_info`, `hcp_job_types` are populated
4. **Existing Client**: Call from a known customer's phone — verify agent greets with context
5. **Availability**: Ask about available slots — verify agent says code-phrase and returns slots grouped by date
6. **Booking**: Complete a booking conversation — verify job and appointment appear in HouseCallPro
7. **Cancellation**: Request cancellation — verify appointment removed from HouseCallPro

If no action occurs:

- Ensure `hcp_api_key` is set correctly and has appropriate permissions
- Verify `project_attributes_settings_newo_tools` is not empty (should contain 3 tools)
- Verify static data was cached (check `hcp_company_employees` is not empty)
- Confirm the relevant feature flags are enabled (`hcp_enable_booking`, `hcp_enable_slot_check`, `hcp_enable_cancellation`)
- For POC method: verify at least one employee has Point of Contact permission enabled in HouseCallPro
- For job_type method: verify employees have relevant tags/skills configured; employees with tag "Other" serve as universal fallback

- For zip_code method: verify employees have zip code tags matching the service area
- Check that `hcp_booking_duration` is set to the intended number of minutes. Legacy values `1..12` are still interpreted as hours for backward compatibility
- Check that `hcp_show_for_days` is a reasonable value
- Re-publish the project if the API key was changed (publish twice on first setup to ensure custom tool registration)

> *Note: This integration performs real operations in HouseCallPro. Jobs, appointments, and customers created or cancelled through the agent are reflected in the live HouseCallPro system.*

5. FAQ

Q: How does the custom tooling work? A: Instead of standard NAF booking tools, this integration registers 3 custom tools in `project_attributes_settings_newo_tools`. Each tool is triggered when the agent says a specific code-phrase (defined in `conditions.agentPromised`). The tool receives the full conversation history (`arguments.memory`), and NSLG extraction skills use LLM to extract structured parameters. Results are returned via `urgent_message` events.

Q: What are the three employee selection methods and when should I use each? A: **POC** (default) assigns the Point of Contact employee — best for single-technician businesses or when one person handles all dispatch. **Job Type** uses AI to match the customer's service description to employee skills/tags — best for multi-specialty teams. Employees with a generic tag like "Other" are treated as universal fallbacks. **Zip Code** matches by service area — best for geographically distributed teams. Set via `hcp_auto_employee_type`.

Q: How does the availability check work? A: The integration sends **parallel requests** to `GET /company/schedule_availability/booking_windows` — one per employee (max 5). Each employee's slots are stored in a separate persona attribute (`hcp_slots_{employee_id}`) to avoid race conditions. Once all responses are collected, slots are merged, sorted by time, and a **slot-to-employee mapping** (`hcp_slot_employee_map`) is saved in persona. This allows the booking flow to assign the correct employee based on the time slot the user selects.

Q: How does the AI line item matching work? A: When `hcp_auto_add_line_items` is enabled, the integration uses Gemini 2.5 Flash to match the customer's service description against the `hcp_line_items` catalog and `hcp_job_types`. The AI selects the most relevant items and attaches them to the job for accurate quoting.

Q: What customer information is required for booking? A: First name, phone number (E.164 format), and full service address (street, city, state, zip, country) are required. Last name defaults to the first name if not provided. Email is optional. Parameters are extracted from conversation memory by an NSLG skill.

Q: How does cancellation work without structured payload? A: When an appointment is booked, `booking_id` and `job_id` are saved to persona attributes (`hcp_last_booking_id`, `hcp_last_job_id`). The cancellation flow reads these IDs from persona — no need to extract from conversation memory.

Q: How often is static data refreshed? A: By default every 24 hours (`hcp_static_data_update_interval` = 86400 seconds). The cache is checked on each session start. You can trigger a manual refresh via the `hcp_update_static_data` event or by re-publishing the project.

Q: What is the default appointment duration? A: 120 minutes, configurable via `hcp_booking_duration`. The end time is calculated as start time + duration in minutes. For backward compatibility, legacy values

`1..12` are still treated as hours and converted to minutes. Times are converted from the business timezone to UTC for the API.

Q: Why do I need to publish twice on first setup? A: On the first publish, the setup persona is created. On the second publish, `_setupToolsSkill` can find the persona's system actors and send the `generalmanageragent_update_newo_tools` event for hot-reload. The custom tools attribute is always written regardless, but the event ensures ConvoAgent picks them up immediately.

Q: What happens if no employees match the selected method? A: The integration has a 3-level fallback: (1) primary method (job_type/zip_code), (2) POC fallback, (3) all employees with `can_be_booked_online=true`. If all return empty, availability returns no slots and the agent informs the customer.

Q: How does the slot-to-employee mapping work? A: When availability is checked, each employee gets a separate API request. The resulting slots are merged into a single list, and a mapping of `"date time" → employee_id` is stored in persona as `hcp_slot_employee_map`. When the user selects a time slot and confirms booking, `_getEmployeeSkill` looks up the employee from this map using the `bookingDateTime`. If no match is found (e.g., manual date entry), it falls back to `selected_employee_id`.

Q: How is double booking prevented? A: A `hcp_booking_in_progress` flag is set in persona when booking starts. If a second booking request arrives while the first is still processing (e.g., agent repeats the code-phrase), it is silently skipped. The flag is cleared on both success and error.

Q: What happens if the customer provides a different address than what's in their HCP profile? A: The integration compares the provided address against all stored addresses by **street + zip code** (case-insensitive). If no match is found, the new address is automatically added to the customer's profile, and the job notes include `"Address updated: {new address}"` so dispatchers are aware.